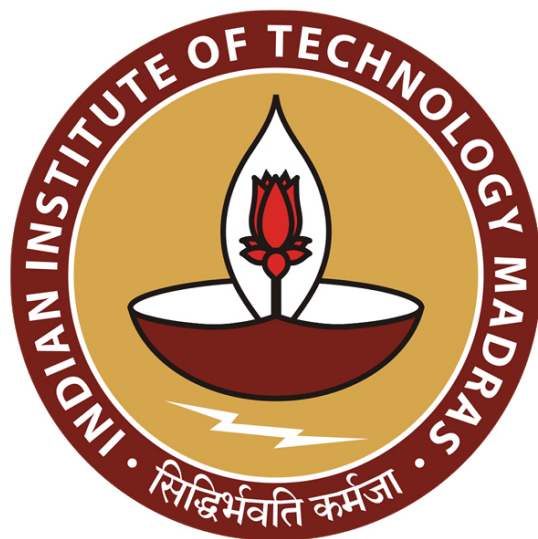




IIT Madras
ONLINE DEGREE

Database Management Systems
Professor Partha Pratim Das
Department of Computer Science & Engineering
Indian Institute of Technology Kharagpur
Concurrency Control/1

Welcome to Module 49 of Database Management Systems course in the IIT Madras online BSc program. We have been discussing about transactions, particularly, serializability, recoverability and so on.

(Refer Slide Time: 00:34)

Module Recap

- With proper planning, a database can be recovered back to a consistent state from inconsistent state in the face of system failures. Such a recovery is done via cascaded or cascadeless rollback
- View Serializability is a weaker serializability system for better concurrency. However, testing for view serializability is NP complete

Database Management Systems

Partha Pratim Das

On that, we had seen that a transaction, a database can be recovered to a consistent state from an inconsistent state due to failure by cascaded or cascadeless rollback. And we have also discussed about view serializability, as a weaker serializability condition for better concurrency. However, testing for it was is NP-complete, so we saw some heuristic methods of checking for it.

(Refer Slide Time: 01:06)

IIT Madras
Module Objectives

- Concurrency Control through design of serializable schedule is difficult in general. Hence we take a look into locking mechanism and Lock-Based Protocols
- We need to understand how locks may be implemented

Database Management Systems Parth Pratim Das

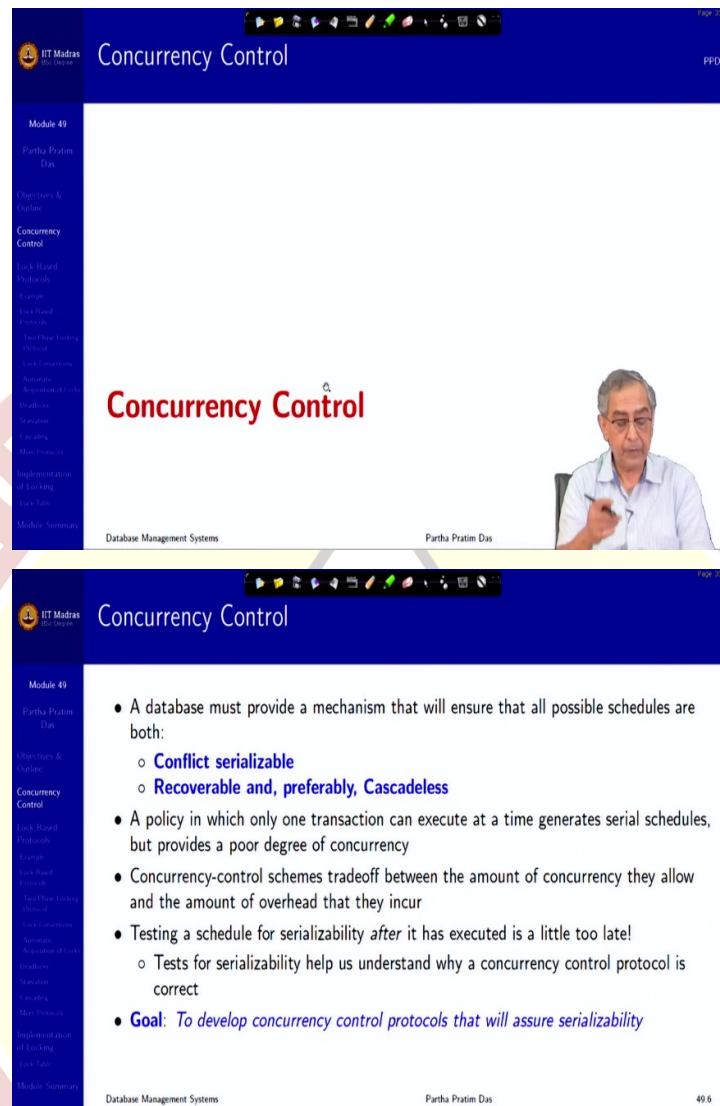
IIT Madras
Module Outline

- Concurrency Control
- Lock-Based Protocols
- Implementing Locking

Database Management Systems Parth Pratim Das

Now, moving forward, the what you are left with using conflict or view serializability and recoverability may not be adequately concurrent schedules. So, we need to manage the concurrency, control the concurrency through the design of the schedule. You cannot just predict that beforehand that this is a schedule which is best, but can we make it, so that, the schedule as it goes would be able to take concurrency as and when it comes while maintaining all the ACID properties and also ensuring fair amount of recoverability. And that is what leads us to locking mechanism or the lock-based protocol as we will see.

(Refer Slide Time: 02:00)



The image displays two screenshots from a video lecture titled "Concurrency Control" by Parthiv Pratin Das at IIT Madras. The top screenshot shows the title slide with the text "Concurrency Control" in red. The bottom screenshot shows a slide with the following bullet points:

- A database must provide a mechanism that will ensure that all possible schedules are both:
 - **Conflict serializable**
 - **Recoverable and, preferably, Cascadeless**
- A policy in which only one transaction can execute at a time generates serial schedules, but provides a poor degree of concurrency
- Concurrency-control schemes tradeoff between the amount of concurrency they allow and the amount of overhead that they incur
- Testing a schedule for serializability *after* it has executed is a little too late!
 - Tests for serializability help us understand why a concurrency control protocol is correct
- **Goal:** *To develop concurrency control protocols that will assure serializability*

So, concurrency control, if we look at that there has to be a mechanism that will ensure that all possible schedules are, both conflict serializable because that can be tested in polynomial time and recoverable preferably cascade less recoverable. So, this can be ensured by a policy that once transaction executes at a time, which will always give serial schedules, but it will have a very poor degree of concurrency, very bad throughput. So, the concurrency control schemes trade off somehow dynamically, the amount of concurrency they will allow, and the amount of overhead that you incur for that. For example, testing a schedule for serializability after it has executed, it is certainly too late. I mean, it does not help. So, the goal that we are going to pursue is to develop concurrency control protocols that will assure serializability, not pre-designed ones.

(Refer Slide Time: 03:25)

Concurrency Control (2)

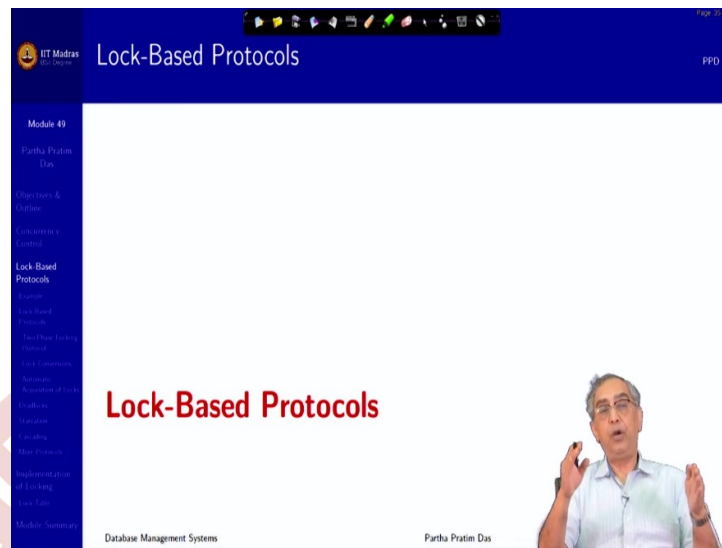
- One way to ensure isolation is to require that data items be *accessed in a mutually exclusive manner*; that is, while one transaction is accessing a data item, no other transaction can modify that data item
 - Should a transaction hold a lock on the whole database
 - ▷ Would lead to strictly serial schedules – very poor performance
- The most common method used to implement locking requirement is to allow a transaction to access a data item only if it is currently holding a **lock** on that item

Database Management Systems | Partho Pratim Das

Now, one way to do that, and I am sure, you have come across this idea when you did operating system in a little different way maybe in terms of semaphores and critical sections and stuff like that is similar that one way to ensure isolation. What is the basic violation of isolation? That the same data item if it is touched by two transactions at the same time then the updates to that may be conflicting.

So, what if I, can guarantee that a data item will always be accessed in a mutually exclusive manner? So, that when one is making a change, the other should not be allowed to make a change. Now, naturally you can enforce that to hold a lock on the entire database. Transaction 1 is working, nobody can be work on that database, that will again lead to serial schedules, but very poor performance. So, we would like to associate locks specifically with data items, and that is the basic idea, to provide exclusive access mechanism. And this whole gamut of strategies, which let you do this is called the lock-based protocol.

(Refer Slide Time: 04:48)



IT Madras
Page 25/31

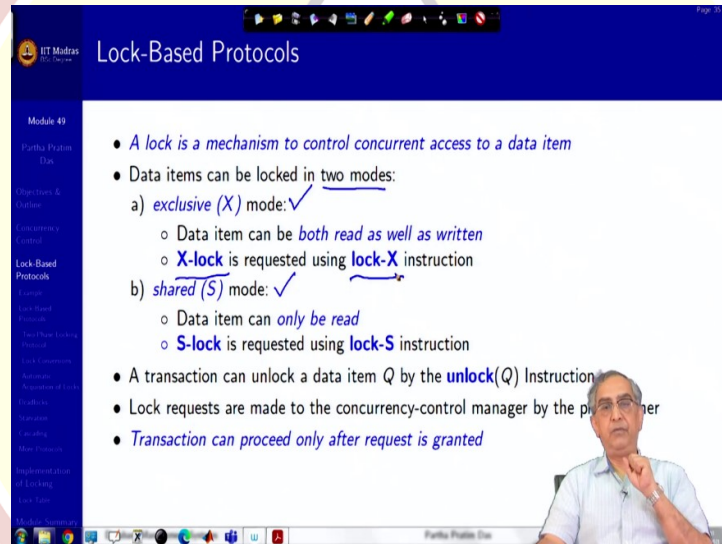
Lock-Based Protocols

Module 49
Parthiv Pratim Das

Database Management Systems

Parthiv Pratim Das

Lock-Based Protocols



IT Madras
Page 26/31

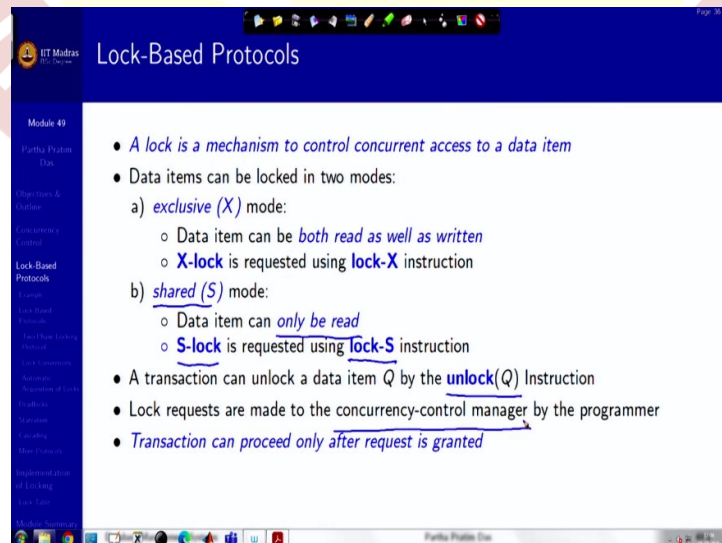
Lock-Based Protocols

Module 49
Parthiv Pratim Das

- A lock is a mechanism to control concurrent access to a data item
- Data items can be locked in two modes:
 - a) exclusive (X) mode: ✓
 - Data item can be both read as well as written
 - X-lock is requested using lock-X instruction
 - b) shared (S) mode: ✓
 - Data item can only be read
 - S-lock is requested using lock-S instruction
- A transaction can unlock a data item Q by the unlock(Q) Instruction
- Lock requests are made to the concurrency-control manager by the programmer
- Transaction can proceed only after request is granted

Database Management Systems

Parthiv Pratim Das



IT Madras
Page 26/31

Lock-Based Protocols

Module 49
Parthiv Pratim Das

- A lock is a mechanism to control concurrent access to a data item
- Data items can be locked in two modes:
 - a) exclusive (X) mode:
 - Data item can be both read as well as written
 - X-lock is requested using lock-X instruction
 - b) shared (S) mode:
 - Data item can only be read
 - S-lock is requested using lock-S instruction
- A transaction can unlock a data item Q by the unlock(Q) Instruction
- Lock requests are made to the concurrency-control manager by the programmer
- Transaction can proceed only after request is granted

Database Management Systems

Parthiv Pratim Das

So, what do you do in the lock-based protocol? A lock is a mechanism, which controls the concurrent access to a data item. So, you can lock a data item in two ways. I mean, that is in general. I mean, there are several kinds of locks, which has several other modes, but the simplest form is two modes. What are they? One is exclusive and another shared.

In the exclusive mode, if you hold a lock in the exclusive mode, you can both read as well as write. Read as well as write. And we normally use X-lock command to request to have a using a lock-X instruction. I will say lock-X data item to get a X-lock that is exclusive lock I can do the read-write both.

The other is a shared lock where I can only read. See, two transactions cannot read at the write at the same time or it is also not possible that one is writing another is reading, the data may be inconsistent, as because you may read before the write or you may read after the write. But two transactions or more can read from the same data item at the same time, so they can share and more often you read, so you have a S-lock which you can get by using the lock-S instruction.

Now, once you have a lock, you have to explicitly provide unlock on that data item to remove that lock, otherwise others will not be able to do anything. So, these are made by the concurrency control manager. So, a transaction always have to request, and they will be able to proceed only if it has got its lock request granted.

(Refer Slide Time: 07:18)

The slide is titled "Lock-Based Protocols (2): Lock Compatibility Matrix". It contains two bullet points: "Lock-Compatibility Matrix: A lock compatibility matrix is used which states whether a data item can be locked by two transactions at the same time" and "Full compatibility matrix". Below these is a table for the full compatibility matrix. The table has rows for "State of the lock" (Unlock, Shared, Exclusive) and columns for "Lock request type" (Shared, Exclusive). The values are: (Unlock, Shared) Yes, (Unlock, Exclusive) Yes, (Shared, Shared) Yes, (Shared, Exclusive) No, (Exclusive, Shared) No, (Exclusive, Exclusive) No. Below this is another bullet point: "Abbreviated compatibility matrix". Below that is a smaller table with the same structure, but with blue checkmarks in the (Shared, Shared) and (Exclusive, Exclusive) cells. A lecturer is visible in the bottom right corner of the slide.

State of the lock	Lock request type	
	Shared	Exclusive
Unlock	Yes	Yes
Shared	Yes	No
Exclusive	No	No

State of the lock	Lock request type	
	Shared	Exclusive
Shared	Yes	No
Exclusive	No	No

Lock-Based Protocols (2): Lock Compatibility Matrix

- **Lock-Compatibility Matrix:** A lock compatibility matrix is used which states whether a data item can be locked by two transactions at the same time
- Full compatibility matrix

State of the lock	Lock request type	
	Shared	Exclusive
Unlock	Yes	Yes
Shared	Yes	No
Exclusive	No	No

- Abbreviated compatibility matrix

State of the lock	Lock request type	
	Shared	Exclusive
Shared	Yes	No
Exclusive	No	No

So, we talk about lock-compatibility matrix. For example, what kind of locks can be held at the same time. So, if you look at the lowered one, a shared lock can be held along with a shared lock because you can read multiple transactions, can read from it. But certainly, a shared lock cannot be held with an exclusive lock, that is, you cannot read when somebody is trying to write or vice versa or two people cannot write. So, this is, yes, but these three are incompatible.

So, which means, that at a time any exclusive lock can be held on a data item by only one transaction, whereas, a shared lock can be held by multiple transactions. Now, naturally when you include unlock since you are unlocking removing this it does not matter what is the lock that is already held on that data item. It can do for shared as well as for exclusive.

(Refer Slide Time: 08:23)

Lock-Based Protocols (3)

- **Requesting for / Granting of a Lock**
 - A transaction may be granted a lock on an item if the requested lock is compatible with locks already held on the item by other transactions
- **Sharing a Lock**
 - Any number of transactions can hold shared locks on an item
 - But if any transaction holds an exclusive lock on the item no other transaction may hold any lock on the item
- **Waiting for a Lock**
 - If a lock cannot be granted, the requesting transaction is made to wait till all incompatible locks held by other transactions have been released
- **Holding a Lock**
 - A transaction must hold a lock on a data item as long as it accesses that item
- **Unlocking / Releasing a Lock**
 - Transaction T_i may unlock a data item that it had locked at some earlier point
 - It is not necessarily desirable for a transaction to unlock a data item immediately after its final access of that data item, since serializability may not be ensured

So, what is a lock-based protocol? That you request for or granting of a lock to the lock manager, so transaction requests for it, and if that lock can be given the transaction will be given the lock. So, for a shared one, for sharing a lock, any number of transaction can hold the shared lock. But if any one transaction holds an exclusive lock, then no other transaction can hold any lock on that.

So, under such cases, you have to wait for the lock till all shared locks or the exclusive lock has been released by the respective transaction. You have to hold on to the lock, as long as you want to access the item otherwise the whole protocol will be violated. And once you are done, so, then you must unlock that data item. But there is a caveat to it.

It is not necessarily always desirable we will see why that you release the lock on a data item, as soon as you are done with that data item. Because there may be some other logical conditions prevalent with other data items as well. But these are the basic actions you can request get a grant. You can share a shared lock. If you do not get a request granted you have to wait for the lock. Once you get you have to hold on to the lock and then finally unlock or release the lock.

(Refer Slide Time: 10:02)

Lock-Based Protocols: Example: Serial Schedule

- Let A and B be two accounts that are accessed by transactions T_1 and T_2 .
 - Transaction T_1 transfers \$50 from account B to account A
 - Transaction T_2 displays the total amount of money in accounts A and B , that is, the sum $A + B$
- Suppose that the values of accounts A and B are \$100 and \$200, respectively
- If these transactions are executed serially, either as T_1, T_2 or the order T_2, T_1 then transaction T_2 will display the value \$300

Transaction Schedules:

T_1 :

- lock-X(B); ✓
- read(B); ✓
- $B := B - 50$; ✓
- write(B); ✓
- unlock(B); ✓
- lock-X(A); ✓
- read(A); ✓
- $A := A + 50$; ✓
- write(A); ✓
- unlock(A); ✓

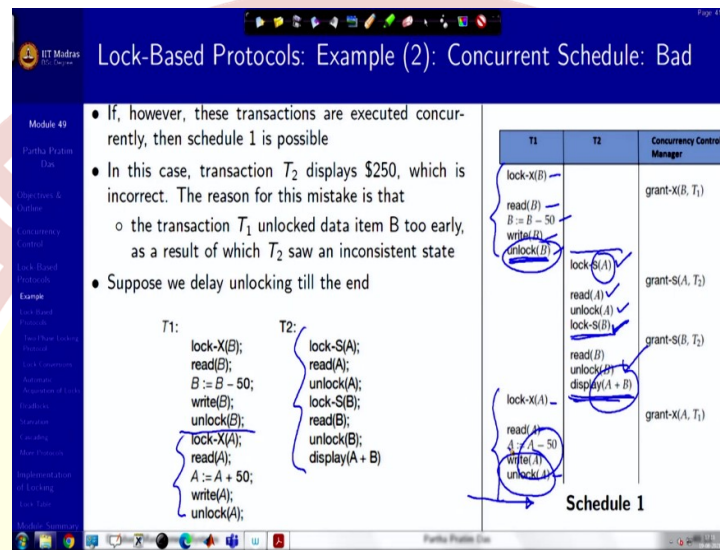
T_2 :

- lock-S(A); ✓
- read(A); ✓
- unlock(A); ✓
- lock-S(B); ✓
- read(B); ✓
- unlock(B); ✓
- display($A + B$)

So, here is an example. So, these are two accounts being accessed by two transactions A and B . Transaction A transfers \$50 transaction B prints the value $A + B$. So, it locks B reads B deducts 50 writes B unlock B . So, during that time nobody can touch B , which is what we wanted to do. Similarly, lock A read A increment by 50 add 50 write backs unlock, nobody can touch. And what this does? This has to print the value of $A + B$ so it is not going to change.

So, it uses a shared lock, reads, unlocks, shared lock to read B unlock display the value. If you execute them serially, either as T1 followed by T2 or T2 followed by T1 you will always get the correct value of 300. If you do T1 followed by T2 then you will have 150 + 150. If you do before T2 first and then T1 you will have 100 + 200 in what case the result will be 300, which is correct.

(Refer Slide Time: 11:30)



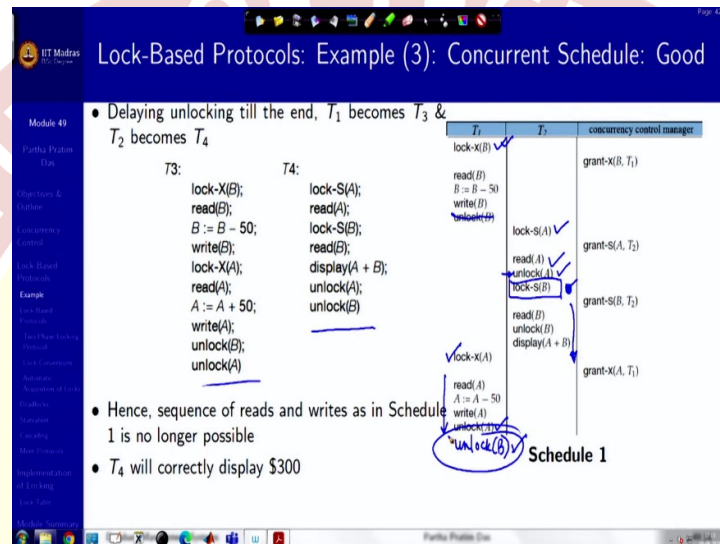
Now, let us see, what will happen in terms of concurrency. Suppose, we make this concurrent, so what it do is in T1, we have up to this much here, and then we have T2 here. And then again, we have the remaining part of T1 here. And we are following the protocol strictly. Before reading and changing it has taken an exclusive lock read, computed, changed and then unlocked because it is done. Similarly, it has done the same for A you have seen this works. But in spite of that, what will this print?

Since it will get the lock on A easily because T1 has not done anything. It reads, unlock, say that is fine. Now, when it ask for lock on B since T1 has already unlocked B it will get this shared lock, and it will be able to read that value of B which is 50 less, which is the final value true.

But just think about it, the value of A that it has will get changed in future. So, at this point, if you remember the color diagram, we had shown couple of modules back of green, yellow and red at this point the database is in inconsistent, transitively inconsistent state. And because this B has been unlocked transaction T2 can read it here, and what it will display quite naturally is 250, 100 + 150.

Whereas, at the end here, when you reach here the database is inconsistent because it is back to 300, but this transaction will show a wrong value. So, that is what I meant that though you should release the lock as soon as at the earliest as you can, but you have to be careful in terms of the logic. Here, you are not doing anything with B, but the point is, if you allow someone else to read this B before you have taken care of the A, there will be inconsistency reflected in that transaction. So, this is a bad way of doing that transaction.

(Refer Slide Time: 14:02)



So, what we should do is, this. Instead of unlock B here, we should unlock B here. What will happen if we unlock B here? If we unlock B here, then this has been done write has been done, you are still holding the lock on B, this will hold a shared lock on A, it will read A, unlock A, it will try to hold a shared lock on B.

So, it has, T2 has read A in a concurrent fashion. It tries to hold a shared lock to read B, but the lock is held here exclusively. So, it cannot get it, it will have to wait, T2 has to wait and since it has released lock A already you will be able to get this unlock A. You will be able to get this exclusive lock on A. T1 would be able to get that and T1 can proceed, T1 releases.

Then finally, when it releases B, the lock on B only then T2 will get it. So, T2 has been waiting here. And after this, T2 will again be able to proceed where you are back to a consistent state. So, when you do this in terms of T3, T4 here you get a consistent result. And this needs quite a bit of logical thinking for the transaction writer to make sure that even though it does not appear to be necessary, but because of what other transactions could do, you will have to delay the unlock of this particular data item.

(Refer Slide Time: 16:08)

Lock-Based Protocols. Example (4). Concurrent Schedule: Deadlock

- Given, T_3 and T_4 , consider Schedule 2 (partial)
- Since T_3 is holding an exclusive mode lock on B and T_4 is requesting a shared-mode lock on B , T_4 is waiting for T_3 to unlock B
- Similarly, since T_4 is holding a shared-mode lock on A and T_3 is requesting an exclusive-mode lock on A , T_3 is waiting for T_4 to unlock A
- Thus, we have arrived at a state where neither of these transactions can ever proceed with its normal execution
- This situation is called **deadlock**
- When deadlock occurs, the system must roll back one of the two transactions.
- Once a transaction has been rolled back, the data items that were locked by that transaction are unlocked.
- These data items are then available to the other transaction, which can continue with its execution.

Schedule 2

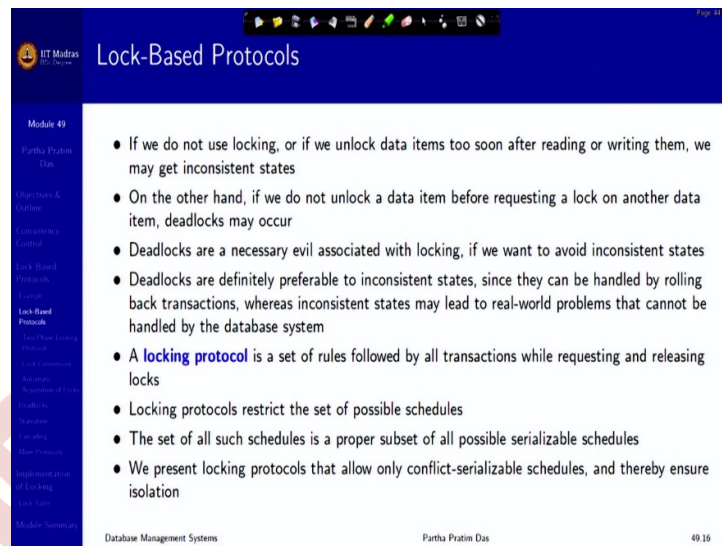
T_3	T_4
lock-X(B)	
read(B)	
$B := B - 50$	
write(B)	
	lock-S(A)
	read(A)
	lock-S(B)
	read(B)
	display($A + B$)
	unlock(A)
	unlock(B)
lock-X(A)	
read(A)	
$A := A + 50$	
write(A)	
unlock(B)	
unlock(A)	

Now, log-based, so that is how to use that lock-based protocol. Now, lock-based protocols can land you in other problems. For example, think about here, you have an exclusive lock X, you have a shared lock S A, you have got that. So, now at this point you need an exclusive lock here. Because now you have known that well, as soon as you are done you should not release that lock, so you are trying to hold on to the lock so unlocks are all at the end.

So, T_3 holds the lock on B exclusive, T_4 holds the lock on A , which is shared then T_3 can go up to this where it needs an exclusive lock on A , it cannot get because T_4 has a shared lock on it, they are not compatible so that it will have to wait here. T_4 needs a shared lock on B to be able to read that value, it cannot get because it is T_3 is locked at here, so it has to.

So, both these transactions will wait at these respective points with no resolution. T_3 will wait on the lock on A to be released, but that can be done only if T_4 gets the lock on B , which can be obtained only if T_3 releases the lock on B , which will happen only if T_3 gets the lock on A circular logic. So, will, none of the transactions will be able to work this is called deadlock. It is very similar to the deadlock in operating system. So, somehow one of the transactions will have to be thrown back rolled back locks released so that the deadlock can be broken.

(Refer Slide Time: 18:05)



Lock-Based Protocols

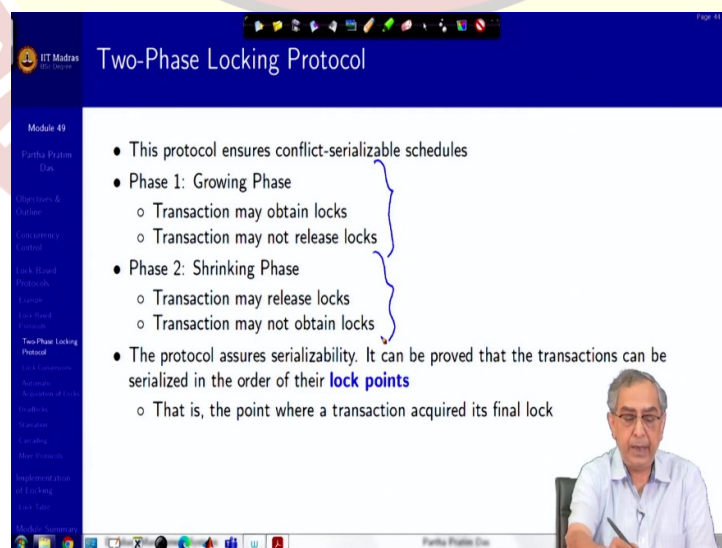
- If we do not use locking, or if we unlock data items too soon after reading or writing them, we may get inconsistent states
- On the other hand, if we do not unlock a data item before requesting a lock on another data item, deadlocks may occur
- Deadlocks are a necessary evil associated with locking, if we want to avoid inconsistent states
- Deadlocks are definitely preferable to inconsistent states, since they can be handled by rolling back transactions, whereas inconsistent states may lead to real-world problems that cannot be handled by the database system
- A **locking protocol** is a set of rules followed by all transactions while requesting and releasing locks
- Locking protocols restrict the set of possible schedules
- The set of all such schedules is a proper subset of all possible serializable schedules
- We present locking protocols that allow only conflict-serializable schedules, and thereby ensure isolation

Database Management Systems | Partha Pratim Das | 49/16

So, deadlock is a basic problem of this log-based protocols, but still, we would prefer it because we have two conflicting requirements. Now, either if we do not if we do not ensure the isolation through the locks, we can potentially get into inconsistent state. If you try to insure through the locks then deadlock may happen.

Now, out of these two things, deadlock is an evil, but deadlock is still preferable because we can still undo or rollback certain transactions, get out of the deadlock and try it again. But if the database gets into inconsistent state, then there are effects in the real world which we have no way to hold back.

(Refer Slide Time: 18:59)



Two-Phase Locking Protocol

- This protocol ensures conflict-serializable schedules
- Phase 1: Growing Phase
 - Transaction may obtain locks
 - Transaction may not release locks
- Phase 2: Shrinking Phase
 - Transaction may release locks
 - Transaction may not obtain locks
- The protocol assures serializability. It can be proved that the transactions can be serialized in the order of their **lock points**
 - That is, the point where a transaction acquired its final lock

Database Management Systems | Partha Pratim Das | 50/16

So, we would try to see how you know these possibilities of rollback can be minimized. So, one way of doing that is, to follow what is called a Two-Phase protocol. So far, we are saying that you need at a time you take it, you are done with the data item some time you release it. But no, here we are saying that, a transaction is, will be, a schedule will be conflict serializable provided the transactions they are in follow a two-phase locking protocol in that in the growing phase the transaction may obtain locks but it does not release any lock. And in the shrinking phase, it may release locks but may not obtain any lock. If it follows this discipline, then this protocol will assure serializability. So that is a nice property.

(Refer Slide Time: 20:02)

Two-Phase Locking Protocol (2)

- There can be conflict serializable schedules that cannot be obtained if two-phase locking is used
- However, in the absence of extra information (that is, ordering of access to data), two-phase locking is needed for conflict serializability in the following sense:
 - Given a transaction T_i that does not follow two-phase locking, we can find a transaction T_j that uses two-phase locking, and a schedule for T_i and T_j that is not conflict serializable

Database Management Systems | Partha Pratim Das

So, anything, which any schedule which follows the two-phase locking protocol is conflict serializable, but the reverse is not true. There can be conflict serializable schedule that cannot be obtained by two-phase locking. But, if we do not have enough data we will have to go with this.

(Refer Slide Time: 20:28)

Lock Conversions

- Two-phase locking with lock conversions:
 - First Phase:
 - ▷ can acquire a lock-S ✓
 - ▷ can acquire a lock-X on item ✓
 - ▷ can convert a lock-S to a lock-X (upgrade) ✓
 - Second Phase:
 - ▷ can release a lock-S ✓
 - ▷ can release a lock-X ✓
 - ▷ can convert a lock-X to a lock-S (downgrade) ✓
- This protocol assures serializability. But still relies on the programmer to insert the various locking instructions

Now, in many cases the two-phase locking is actually extended in the growing phase, as well as in the shrinking phase by including conversion. This two we talked about that it can acquire a lock so it can acquire an S-lock, it can acquire a X-lock or it can convert an S-lock to a X-lock which is upgrade.

So, instead of releasing it, you are holding an S-lock shared lock, you request for an upgrade to a exclusive lock. So, if nobody else is holding a shared lock on that, you will be given that, if somebody else is also holding, then you will not be. So, this is also what you can do in the growing phase.

Similarly, in the shrinking phase, you release the S-lock, you release the exclusive lock or you convert a exclusive lock to a shared lock. So, in the exclusive lock you had just alone holding the lock, you downgrade that to a shared lock, so that others can also hold shared lock that this protocol assures serializability and but certainly the programmer will have to finally insert the different locking instructions.

(Refer Slide Time: 21:52)

Automatic Acquisition of Locks: Read

- A transaction T_i issues the standard read/write instruction, without explicit locking calls
- The operation $\text{read}(D)$ is processed as:
if T_i has a lock on D
then
 $\text{read}(D)$ ✓
else begin
 if necessary, wait until no other transaction has a lock-X on D
 grant T_i a lock-S on D ;
 $\text{read}(D)$ ✓
end

So, another alternative that many systems actually support the programmer with, is by automatic acquisition of locks. And because you know that if you have to read then you need a shared lock. So, why do you tell the programmer to get it, you can just get that. So, the operation says that read will be, if the transaction has a lock on this item, then read it. if it is, then wait till no other transaction has exclusive lock on it. The others may have shared lock because they are going to read. Then T_i will be granted a lock S, and then you read it. So, you just write-read this will happen in the background to ensure the isolation.

(Refer Slide Time: 22:51)

Automatic Acquisition of Locks: Write

- $\text{write}(D)$ is processed as:
if T_i has a lock-X on D
then
 $\text{write}(D)$ ✓
else begin
 if necessary, wait until no other transaction has any lock on D , ✓
 if T_i has a lock-S on D
 then
 upgrade lock on D to lock-X ✓
 else
 grant T_i a lock-X on D
 $\text{write}(D)$ ✓
 end;
• All locks are released after commit or abort

Similarly, for write, little bit more complex, but very intuitive. For write, you need an exclusive lock. If you have that go ahead and write. If you do not, you have to wait until no

one else has any lock, because you are going to write, so you are going to ask for exclusive lock.

Now, if you are the only person. This transaction, T_i is the only transaction which has a shared lock then you will request for an upgrade. If not, then you will request for a grant of the exclusive lock. These are just the just enumerating the possibilities, so that, before you write you necessarily hold a exclusive lock. Before you read you necessarily hold a shared lock, and you relieve the programmer from the burden of putting it at every place.

(Refer Slide Time: 23:51)

Deadlocks

- Two-phase locking *does not* ensure freedom from deadlocks

T_3 :

```
lock-X(B);
read(B);
B := B - 50;
write(B);
lock-X(A);
read(A);
A := A + 50;
write(A);
unlock(B);
unlock(A);
```

T_4 :

```
lock-S(A);
read(A);
lock-S(B);
read(B);
display(A + B);
unlock(A);
unlock(B);
```

T_3	T_4
lock-x (B) ✓	
read (B)	
B := B - 50	
write (B)	
lock-x (A)	lock-s (A) ✗
	read (A)
	lock-s (B)

- Observe that transactions T_3 and T_4 are two phase, but, in deadlock

So, two-phase locking does not ensure the freedom from deadlocks. You can see the transactions here, they are doing two-phase locking because they are only locking, they are only locking, so both are in the growing phase before they have their shrinking phase. But this will also be a deadlock because this will be granted, this will be granted, this will wait for this and this will wait for this. So, you will have a deadlock again. So, it does not guarantee that.

(Refer Slide Time: 24:35)

Starvation

- In addition to deadlocks, there is a possibility of **Starvation**
- **Starvation** occurs if the concurrency control manager is badly designed. For example:
 - A transaction may be waiting for an X-lock on an item, while a sequence of other transactions request and are granted an S-lock on the same item
 - The same transaction is repeatedly rolled back due to deadlocks
- Concurrency control manager can be designed to prevent starvation
- Starvation is also loosely referred to as **Livelock**

Database Management Systems | Partha Pratim Das

Actually, when you make transactions wait for an exclusive lock then many transactions maybe be waiting. So, you will have to give it to someone and rollback others or keep them waiting. So, it is possible that a particular transaction may not get the lock it is requesting for a long time. Even though, there may be several instances, this is not deadlock, because several others have got, but it is just that your turn is not coming. So, that is called starvation. In a loose sense, it is also referred to as Livelock this is something which exists in operating system again, and these kind of things can also happen with the locking protocol.

(Refer Slide Time: 25:27)

Cascading Rollback

- The potential for deadlock exists in most locking protocols. Deadlocks are a necessary evil
- When a deadlock occurs there is a possibility of cascading roll-backs
- Cascading roll-back is possible under two-phase locking
- In the schedule here, each transaction observes the two-phase locking protocol, but the failure of T5 after the read(A) step of T7 leads to cascading rollback of T6 and T7.

T ₅	T ₆	T ₇
lock-X(A)		
read(A)		
lock-S(B)		
read(B)		
write(A)		
unlock(A)		
	lock-X(A)	
	read(A)	
	write(A)	
	unlock(A)	
		lock-S(A)

Database Management Systems | Partha Pratim Das

And so, the potential of deadlock exists in all locking protocols. And when a deadlock happens, you will have to roll-back one or more transactions, and when that happens, obviously, there is a possibility of a cascading roll-back. This particular example will show you that. Study it, I will not go through each and every study it you will be able to see it very clearly.

(Refer Slide Time: 25:53)

More Two Phase Locking Protocols

- To avoid Cascading roll-back, follow a modified protocol called **strict two-phase locking**
 - a transaction must hold all its exclusive locks till it commits/aborts
- **Rigorous two-phase locking** is even stricter
 - All locks are held till commit/abort. In this protocol transactions can be serialized in the order in which they commit
- Note that concurrency goes down as we move to more and more strict locking protocol

Now, there are ways to avoid cascading roll-back, which is known as a strict two-phase protocol. We have just discussed two-phase protocol, there is a strict two-phase protocol which say that a transaction must hold all its exclusive locks till it commits or aborts. This will guarantee that you do not have cascading,

I mean, if you have, if you can close your eyes and see what cascading roll-back is doing you will understand that because if you are holding on others are not being able to work on that. So, if they are not able to work on that, there may not be transactions which have already done partial work and needs a cascading roll-back.

There is a rigorous one, where all locks not only exclusive ones, but all locks are held till the committed abort has happened. In this protocol, transactions can be serialized in the order in which they commit. So, these are different protocols. I mean, you do not have to really work through each one of them in detail, but just know that these exists. It is important to understand the two-phase protocol and be able to solve problems with that two-phase protocol.

And certainly, one point has to be always kept in mind that the more and more strict you get in terms of the locking protocol, you ensure safer, roll-back and so on, but certainly, your concurrency is going down. This is this direct trade off, it is very easy to see because you are the more strict locking protocol you have you will hold the locks on data items for a longer period. The longer you hold that lock on data items naturally the option for other transactions to work with those data items become less concurrency goes down, that is a simple.

(Refer Slide Time: 28:04)

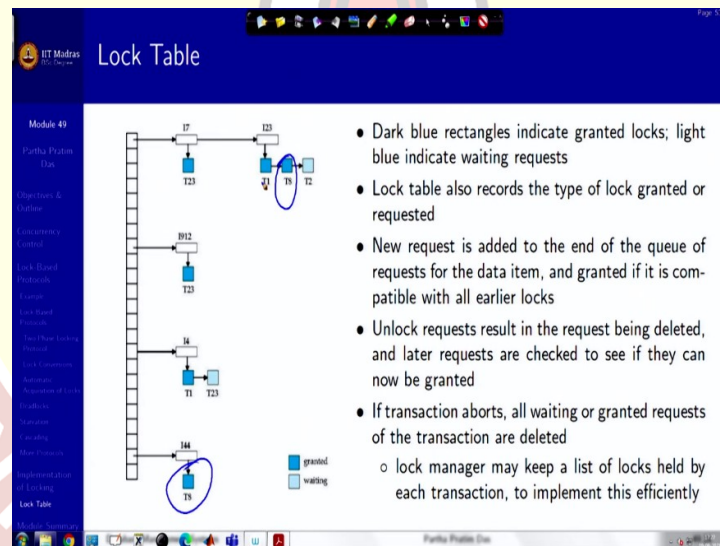


The image displays two screenshots from a video lecture titled "Implementation of Locking" by Partha Pratim Das. The top screenshot shows the title slide with the text "Implementation of Locking" in red. The bottom screenshot shows a list of bullet points explaining the implementation of locking.

- A **lock manager** can be implemented as a separate process to which transactions send lock and unlock requests
- The lock manager replies to a lock request by sending a lock grant messages (or a message asking the transaction to roll back, in case of a deadlock)
- The requesting transaction waits until its request is answered
- The lock manager maintains a data-structure called a **lock table** to record granted locks and pending requests
- The lock table is usually implemented as an in-memory hash table indexed on the name of the data item being locked

Now, in terms of implementation of locking. I mean, I just included that because, but it is very simple straightforward. So, you have a lock manager which manages this, and it is done in terms of lock tables. So, all that you will need is for every data item, you need to remember what are the transactions that are holding locks, what are the types of locks and who are the

(Refer Slide Time: 28:34)



Of course, I mean, how do you do it efficiently when you have a large number of data items and large number of transaction, of course, a good challenge in algorithm and data structure

design, but in principle it is quite easy to do that. You will also have to have references in some other way. For example, if a transaction aborts or if you want to have to rollback a transaction then you need to, this is looks like more like going by data item wise, but that may not be enough. If you have to say roll-back transaction T8 you need to know that these are the two locks already granted to it. So, there has to be other form of reference also available in the lock manager for doing such tasks.

(Refer Slide Time: 30:09)

Module Summary

- Understood the locking mechanism and protocols
- Realized that deadlock is a peril of locking and needs to be handled through rollback

Slides used in this presentation are borrowed from <http://db-book.com/> with kind permission of the authors.
Edited and new slides are marked with "PPD".

Database Management Systems Partha Pratim Das 49/29

So, we come to the end of this module. Thank you very much for your attention. We have discussed quite a bit in terms of the locking protocol and the primary evil that it creates the deadlock. So, we will have to take a little bit more head on with the deadlock in the next module. See you then.